



DOCENTE: Thiago Naves

DATA: 01/06/2026

ALUNOS: Eduardo Andrei Staudt e Brandon Monteiro Donisthorpe

RAs: 2783045 e 2758890

TRABALHO - Matriz Encadeada

1. Objetivo

Este trabalho tem como objetivo desenvolver funções, com base nos conceitos de listas e encadeamento, para simular e manipular uma matriz bidimensional de inteiros utilizando ponteiros. Diferente de uma matriz convencional alocada como vetor de vetores, cada elemento aqui é um nó que mantém ponteiros para os seus quatro vizinhos (cima, baixo, esquerda e direita), de forma que a estrutura possa ser percorrida exclusivamente pelos ponteiros, como em uma lista duplamente encadeada nas duas dimensões.

2. Ferramentas Utilizadas

- Excalidraw - para os desenhos;
- VsCode - para construção do projeto;
- Carbon - para as molduras dos blocos de código da documentação;
- Google Docs - para a escrita da documentação.

3. Estruturas de Dados Utilizadas

struct elemento (Elem)



```
typedef struct elemento Elem;

struct elemento {
    int x;           // linha
    int y;           // coluna
    int valor;       // valor armazenado
    Elem *cima;      // vizinho de cima
    Elem *baixo;     // vizinho de baixo
    Elem *esq;       // vizinho da esquerda
    Elem *dir;       // vizinho da direita
};
```

Representa um nó individual da matriz. Cada nó guarda sua coordenada (x, y), o valor inteiro armazenado e quatro ponteiros para os vizinhos.

struct matriz (Matriz)

```
struct matriz {
    int linhas;       // quantidade de linhas da matriz
    int colunas;      // quantidade de colunas da matriz
    Elem *inicio;     // ponteiro para a coordenada (0,0)
};
```

struct Matriz



seta verde = ponteiro de entrada (m->inicio)

Imagem 1: representação da struct matriz Fonte: Autoria própria feito no excalidraw.

Representa a matriz como um todo. Guarda as dimensões e um único ponteiro para o nó da coordenada (0,0), que é o ponto de partida para qualquer navegação na estrutura, conforme exigido pelo enunciado.

Representação visual da matriz encadeada (3x4):

Matriz 3x4 encadeada

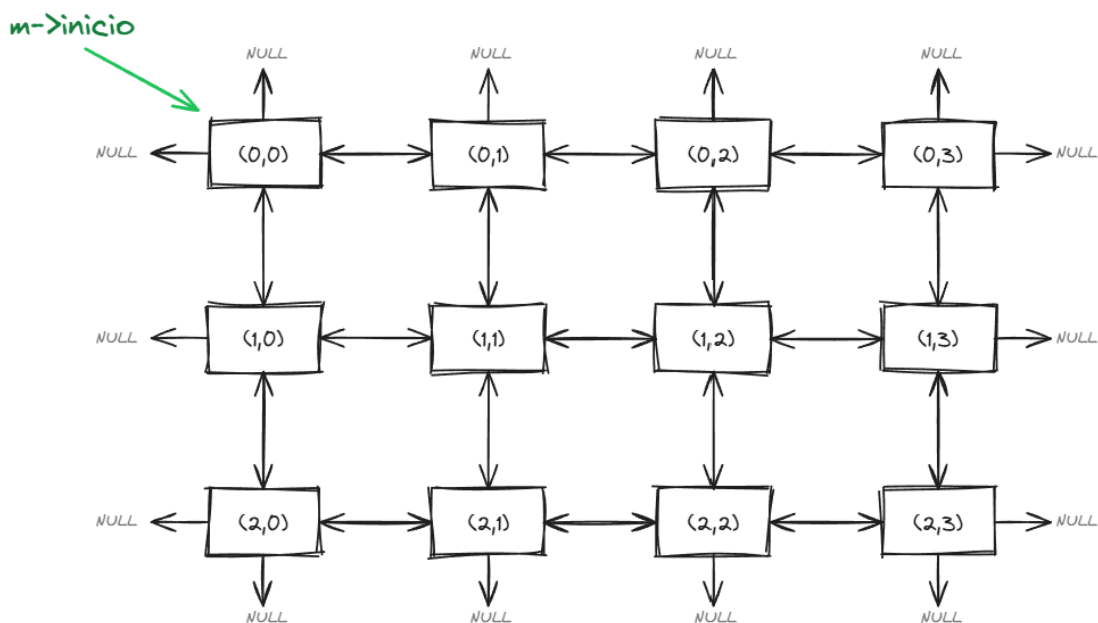




Imagem 1: Representação de um exemplo de matriz 3X4. Fonte: Autoria própria feito no excalidraw.

A matriz é formada por nós ligados por ponteiros. `m->inicio` aponta para o nó (0,0), e a partir dele dá para navegar em qualquer direção com `dir`, `esq`, `cima` e `baixo`. Nos nós de borda, o ponteiro da direção sem vizinho fica NULL: `cima` na primeira linha, `esq` na primeira coluna, `baixo` na última linha e `dir` na última coluna.

4. Funções `matriz.c`

Função `desalocar_matriz`:

```
int desalocar_matriz(Matriz *m) {
    if (m == NULL) return 1;
    Elem *linha = m->inicio;
    while (linha != NULL) {
        Elem *prox_linha = linha->baixo;
        Elem *atual = linha;
        while (atual != NULL) {
            Elem *no = atual;
            atual = atual->dir;
            free(no);
        }
        linha = prox_linha;
    }
    free(m);
    return 0;
}
```

A função percorre a matriz linha por linha, salvando o ponteiro para a próxima linha antes de liberar a atual. Dentro de cada linha, percorre os nós pela direita e chama `free` em cada um. Ao final, libera também a struct `Matriz`. Se receber NULL, retorna 1. Se liberar tudo sem problema, retorna 0.

Função `alocar_matriz`:



```
Matriz* alocar_matriz(int linhas, int colunas) {
    if (linhas <= 0 || colunas <= 0) return NULL;

    Matriz *m = (Matriz*) malloc(sizeof(Matriz));
    if (m == NULL) return NULL;

    m->linhas = linhas;
    m->colunas = colunas;
    m->inicio = NULL;

    Elem *linha_ant = NULL;

    for (int i = 0; i < linhas; i++) {
        Elem *primeiro = NULL;
        Elem *ant = NULL;
        Elem *acima = linha_ant;

        for (int j = 0; j < colunas; j++) {
            Elem *no = (Elem*) malloc(sizeof(Elem));
            if (no == NULL) {
                Elem *atual = primeiro;
                while (atual != NULL) {
                    Elem *prox = atual->dir;
                    free(atual);
                    atual = prox;
                }
                Elem *linha = m->inicio;
                while (linha != NULL) {
                    Elem *prox_linha = linha->baixo;
                    Elem *elem = linha;
                    while (elem != NULL) {
                        Elem *prox_elem = elem->dir;
                        free(elem);
                        elem = prox_elem;
                    }
                    linha = prox_linha;
                }
                free(m);
                return NULL;
            }
            no->x = i;
            no->y = j;
            no->valor = 0;
            no->cima = NULL;
            no->baixo = NULL;
            no->esq = NULL;
            no->dir = NULL;
            if (ant != NULL) {
                ant->dir = no;
                no->esq = ant;
            }
            if (acima != NULL) {
                acima->baixo = no;
                no->cima = acima;
                acima = acima->dir;
            }
            if (j == 0) primeiro = no;
            ant = no;
        }
        if (i == 0) m->inicio = primeiro;
        linha_ant = primeiro;
    }
    return m;
}
```



A função aloca uma matriz encadeada de linhas x colunas nós do tipo Elem. Cada nó guarda sua posição (x, y), valor inicial 0 e quatro ponteiros para os vizinhos (cima, baixo, esq, dir). Os nós são conectados horizontalmente e verticalmente no momento da criação. Se as dimensões forem inválidas ou alguma alocação falhar, retorna NULL. Se tudo der certo, retorna o ponteiro para a matriz criada.

Função inserir_valor:

```
int inserir_valor(Matriz *m, int x, int y, int valor) {  
    if (m == NULL) return 1;  
    if (x < 0 || x >= m->linhas) return 1;  
    if (y < 0 || y >= m->colunas) return 1;  
    Elem *atual = m->inicio;  
    for (int i = 0; i < x; i++) {  
        atual = atual->baixo;  
    }  
    for (int j = 0; j < y; j++) {  
        atual = atual->dir;  
    }  
    atual->valor = valor;  
    return 0;  
}
```

A função valida a matriz e as coordenadas (x, y) antes de fazer qualquer coisa. Se estiver tudo certo, parte do nó (0,0) e caminha pelos ponteiros baixo até a linha x, depois pelos ponteiros dir até a coluna y. Chegando no nó, atualiza o campo valor. Retorna 1 se a matriz for NULL ou a posição estiver fora dos limites, e 0 se a inserção funcionar.

Função buscar_valor:



```
Elem* buscar_valor(Matriz *m, int valor)
{
    if (m == NULL) return NULL;
    Elem *linha = m->inicio;
    while (linha != NULL) {
        Elem *atual = linha;
        while (atual != NULL) {
            if (atual->valor == valor) {
                return atual;
            }
            atual = atual->dir;
        }
        linha = linha->baixo;
    }
    return NULL;
}
```

A função faz uma varredura linha a linha pela matriz. Parte do nó (0,0), caminha para a direita usando dir até o fim da linha, depois desce com baixo para a próxima. Se achar um nó com o valor buscado, retorna o ponteiro para ele. Se a matriz for NULL ou o valor não existir, retorna NULL.

Função consultar_valor:

```
Elem* consultar_valor(Matriz *m, int x, int y) {
    if (m == NULL) return NULL;
    if (x < 0 || x >= m->linhas || y < 0 || y >= m->colunas) return NULL;
    Elem *atual = m->inicio;
    for (int i = 0; i < x; i++) {
        atual = atual->baixo;
    }
    for (int j = 0; j < y; j++) {
        atual = atual->dir;
    }
    return atual;
}
```



A função valida a matriz e as coordenadas (x, y) antes de navegar. Se estiver tudo certo, parte do nó (0,0), desce pelos ponteiros baixo até a linha x e anda pelos ponteiros dir até a coluna y. Retorna o ponteiro para o nó encontrado, ou NULL se a matriz for NULL ou a posição estiver fora dos limites.

Função buscar_valor:

```
Elem* buscar_valor(Matriz *m, int valor)
{
    if (m == NULL) return NULL;
    Elem *linha = m->inicio;
    while (linha != NULL) {
        Elem *atual = linha;
        while (atual != NULL) {
            if (atual->valor == valor) {
                return atual;
            }
            atual = atual->dir;
        }
        linha = linha->baixo;
    }
    return NULL;
}
```

A função faz uma varredura linha a linha pela matriz. Parte do nó (0,0), caminha para a direita com dir até o fim da linha e desce com baixo para a próxima. Se encontrar o valor buscado, retorna o ponteiro para o nó. Se a matriz for NULL ou o valor não existir, retorna NULL.

Função imprimir_vizinhos:



```
int imprimir_vizinhos(Matriz *m, int x, int y) {
    Elem *no = consultar_valor(m, x, y);
    if (no == NULL) return 1;
    printf("Vizinhos do no (%d,%d) com valor %d:\n", no->x, no->y, no->valor);
    if (no->cima != NULL)
        printf("Cima: %d\n", no->cima->valor);
    else
        printf("Cima: NULL\n");
    if (no->baixo != NULL)
        printf("Baixo: %d\n", no->baixo->valor);
    else
        printf("Baixo: NULL\n");
    if (no->esq != NULL)
        printf("Esquerda: %d\n", no->esq->valor);
    else
        printf("Esquerda: NULL\n");
    if (no->dir != NULL)
        printf("Direita: %d\n", no->dir->valor);
    else
        printf("Direita: NULL\n");
    return 0;
}
```

A função usa `consultar_valor` para acessar o nó em (x, y). Se a posição não existir, retorna 1. Se o nó for encontrado, imprime os valores dos quatro vizinhos (cima, baixo, esq, dir), ou NULL para os que não existirem. Retorna 0 ao final.

Função deslocar:



```
int deslocar(Elem **atual, int opcao) {  
    if (atual == NULL || *atual == NULL) return 1;  
    Elem *proximo = NULL;  
    if (opcao == 1) {  
        proximo = (*atual)->baixo;  
    }  
    else if (opcao == 2) {  
        proximo = (*atual)->cima;  
    }  
    else if (opcao == 3) {  
        proximo = (*atual)->dir;  
    }  
    else if (opcao == 4) {  
        proximo = (*atual)->esq;  
    }  
    else {  
        return 1;  
    }  
    if (proximo == NULL) {  
        return 1;  
    }  
    *atual = proximo;  
    return 0;  
}
```

A função usa `consultar_valor` para acessar o nó em (x, y). Se a posição não existir, retorna 1. Se o nó for encontrado, imprime os valores dos quatro vizinhos (cima, baixo, esq, dir), ou NULL para os que não existirem. Retorna 0 ao final.

Função `imprimir_matriz`:



```
int imprimir_matriz(Matriz *m) {  
    if (m == NULL) return 1;  
    Elem *linha = m->inicio;  
    while (linha != NULL) {  
        Elem *atual = linha;  
        while (atual != NULL) {  
            printf("%5d ", atual->valor);  
            atual = atual->dir;  
        }  
        printf("\n");  
        linha = linha->baixo;  
    }  
    return 0;  
}
```

Se a matriz for NULL, retorna 1. Caso contrário, percorre linha por linha: dir para avançar nas colunas e baixo para descer para a próxima linha. Os valores são impressos com %5d para manter o alinhamento em colunas. Retorna 0 ao final.

5. Arquivo main.c



```
+++ MENU DE MOVIMENTO +++
1 - Descer
2 - Subir
3 - Ir para direita
4 - Ir para esquerda
5 - Imprimir matriz
0 - Sair
Escolha uma opcao: 1
Desceu para a posicao (1,0), valor = 0

+++ POSICAO ATUAL +++
Estou na posicao (1,0), valor = 0

+++ MENU DE MOVIMENTO +++
1 - Descer
2 - Subir
3 - Ir para direita
4 - Ir para esquerda
5 - Imprimir matriz
0 - Sair
Escolha uma opcao: 4
Nao foi possivel ir para esquerda. Retornou NULL.

+++ POSICAO ATUAL +++
Estou na posicao (1,0), valor = 0

+++ MENU DE MOVIMENTO +++
1 - Descer
2 - Subir
3 - Ir para direita
4 - Ir para esquerda
5 - Imprimir matriz
0 - Sair
Escolha uma opcao: 0
Saindo...
+++ Desalocando matriz +++
Matriz desalocada com sucesso.
```

```
+++ Vizinhos de cada noh +++
Vizinhos do noh (1,1) com valor 50:
Cima:    20
Baixo:   0
Esquerda: 0
Direita: 70
```

```
Vizinhos do noh (0,0) com valor 10:
Cima:    NULL
Baixo:   0
Esquerda: NULL
Direita: 20
```

```
Vizinhos do noh (2,3) com valor 99:
Cima:    0
Baixo:   0
Esquerda: 0
Direita: 0
```

```
+++ Consultando posicoes +++
Valor em (1,1) = 50
Valor em (2,3) = 99
Valor em (9,9) = NULL
```

```
+++ Buscando valores +++
Valor 70 encontrado em (1,2).
Valor -5 encontrado em (2,0).
Valor 1000 encontrado em (4,4).
```

```
+++ Inserindo valores +++
Valor 1000 encontrado em (4,4).
Valor 2000 nao encontrado
+++ Matriz atual +++
10    20    0    0    0
0     50    70    0    0
-5     0    0    99    0
0     0    0    0    0
0     0    0    0  1000
```

Imagens 3, 4, 5 e 6: demonstração dos resultados da main.c no terminal. Fonte: Autoria própria prints do terminal.



O main.c testa as funcionalidades da matriz em sequência. Aloca uma matriz 5x5(podendo ter diversas dimensões, essa foi a escolhida para o teste), insere valores em posições específicas e faz buscas por valores existentes e inexistentes. Depois imprime a matriz completa, consulta posições e exibe os vizinhos de alguns nós. Por fim, abre um menu interativo onde o usuário move o ponteiro atual pelas quatro direções com deslocar. A matriz é desalocada com desalocar_matriz antes de encerrar.

6. Desafios e Dificuldades

Foi enviado o código para correção na IA DeepSeek e ela havia apontado que a função alocar_matriz continha um possível vazamento de memória por falta de tratamento de exceções e limpeza com o free(), pois quando um nó era nulo ele retornava a função e não liberava a memória dos outros nós já criados no "for". Para isso foi criada uma correção no código.

Código antigo:

```
if (no == NULL) return NULL;
```

Implementação para liberar memória dos nós já criados:



```
if (no == NULL) {
    Elem *atual = primeiro;
    while (atual != NULL) {
        Elem *prox = atual->dir;
        free(atual);
        atual = prox;
    }
    Elem *linha = m->inicio;
    while (linha != NULL) {
        Elem *prox_linha = linha->baixo;
        Elem *elem = linha;
        while (elem != NULL) {
            Elem *prox_elem = elem->dir;
            free(elem);
            elem = prox_elem;
        }
        linha = prox_linha;
    }
    free(m);
    return NULL;
}
```

7. Como Compilar e Executar

O programa pode ser compilado em qualquer ambiente com gcc instalado, utilizando os três arquivos entregues:



```
● ● ●  
  
// Pra rodar via terminal:  
gcc main.c matriz.c -o programa  
.\programa.exe
```

A execução produz uma sequência de mensagens demonstrando o funcionamento de cada função implementada.

8. Conclusão

O objetivo do trabalho foi cumprido, a matriz bidimensional funciona só com encadeamento, sem vetores. Cada nó carrega quatro ponteiros (cima, baixo, esq, dir), o que permite navegar em qualquer direção a partir de qualquer posição. Todas as funções obrigatórias foram implementadas e testadas, e duas funções extras foram adicionadas para ajudar na depuração, sendo elas a de imprimir_matriz(imprime a matriz completa) e deslocar(que permite navegar o ponteiro através de nós vizinhos). A parte mais trabalhosa foi a alocação, que precisa conectar os nós verticalmente e horizontalmente ao mesmo tempo sem perder os ponteiros intermediários. O trabalho ajudou a fixar os conteúdos de listas duplamente encadeadas passados em aula, alocação dinâmica e o cuidado com a liberação de memória.